ภาคผนวก ก

Function in Libnet which used in this project

Packet Construction

Packets are constructed modularly. For each protocol layer, there should be a corresponding call to a **libnet_build function**. Depending on your end goal, different things may happen here. For the above IP-layer example, calls to libnet_build_ip() and libnet_build_tcp() will be made. For the link-layer example, an additional call to libnet_build_ethernet() will be made. It is important to note that the ordering of the packet constructor function calls is not significant, it is only necessary that the correct memory locations be passed to these functions. The functions need to build the packet headers inside the buffer as they would appear on the wire and be demultiplexed by the recipient, and this can be done in any order

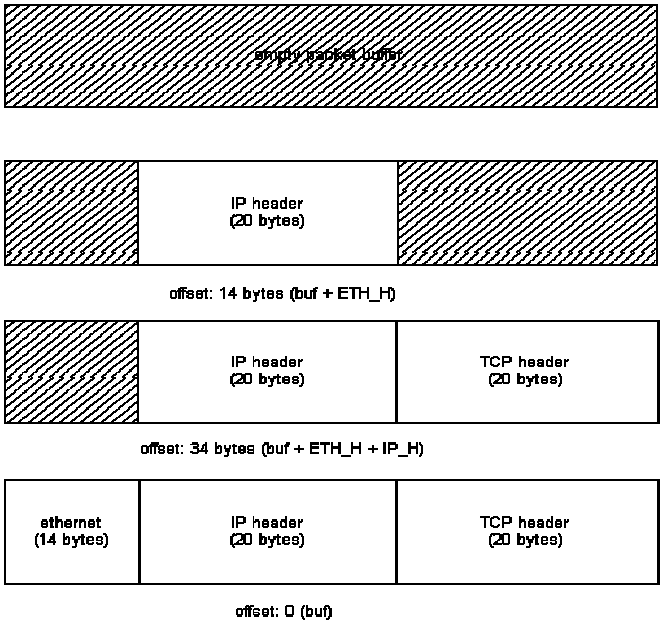

figure 2

libnet_build_ethernet() would be passed the beginning of the buffer with an offset of 0 (as it needs to build an ethernet header at the front of the packet). libnet_build_ip() would get the buffer at a 14

u_long libnet_name_resolve(u_char *ip, u_short use_name);

return value upon success	network-byte ordered IP address
return value upon failure	-1
re-entrant	yes
arguments	1 - human readable IP address or FQDN 2 - use_name flag

libnet_name_resolve() takes a NULL terminated ASCII string representation of an IP address (dots and decimals or, if the usename flag is LIBNET_RESOLVE, a canonical hostname) and converts it into a network-ordered (big-endian) unsigned long value.

int libnet_init_packet(u_short packet_size, u_char **buf);

return value upon success	1
return value upon failure	-1
re-entrant	yes
arguments	1 - desired packet size 2 - address of a u_char pointer

libnet_init_packet() creates memory for a packet (it doesn't so much create memory as it requests it from the underlying operating system via malloc()). Upon success the memory is zero-filled. The function accepts two arguments, the packet size and the address of the pointer to the packet. The packet size parameter may be 0, in which case the library will attempt to guess a packet size for you. Passing in the pointer to a pointer (passing by address) is necessary as we are allocating memory locally. If we instead passed in just a pointer (passing by value) the allocated memory would be lost.

This function is a good example of interface hiding. This function is essentially a malloc() wrapper. By using this function the details of what's really happening are abstracted so that you, the programmer, can worry about your task at hand.

int libnet_write_ip(int socket, u_char *packet, int packet_size);

return value upon success	packet size
return value upon failure	-1
re-entrant	yes
arguments	1 - socket 2 - pointer to the packet 3 - packet size
arguments	1 - maximum size of pseudo-random number desired

libnet_write_ip() writes an IP packet to the network. The first argument is the socket created with a previous call to libnet_open_raw_sock, the second is a pointer to a buffer containing a complete IP datagram, and the third argument is the total packet size. The function returns the number of bytes written upon success or -1 on error (with errno containing the reason).

void libnet_destroy_packet(u_char **buf);

return value upon success	NA
return value upon failure	NA
re-entrant	yes
arguments	1 - address of a u_char pointer

libnet_destroy_packet() is the free() analog to libnet_init_packet. It destroys the packet referenced by 'buf'. In reality, it is of course a simple free() wrapper. It frees the heap memory and points `buf` to NULL to dispel the dangling pointer. The function does make the assertion that `buf` is not NULL. A pointer to a pointer is passed to maintain interface consistency.

u_long libnet_get_prand(int modulus);

libnet_get_prand() generates a psuedo-random number. The range of the returned number is controlled by the function's only argument:

VALUE	DESCSRIPTION
LIBNET_PR2	0 - 1
LIBNET_PR8	0 - 255
LIBNET_PR16	0 - 32767
LIBNET_PRu16	0 - 65535
LIBNET_PR32	0 - 2147483647
LIBNET_PRu32	0 - 4294967295

The function does not fail.

int libnet_seed_prand();

return value upon success	1
return value upon failure	-1
re-entrant	yes
arguments	NA

libnet_seed_prand() seeds the pseudo-random number generator. The function is basically a wrapper to srandom. It makes a call to gettimeofday to get entropy. It can return -1 if the call to gettimeofday fails (check errno). It otherwise returns 1.